Unidad II — Exposición por equipos

GitHub Actions: Detectar errores antes de que lleguen al usuario

Materia: Fundamentos de Arquitectura de Software y Desarrollo Backend

Integrantes del equipo:

David Morales

Edward Chay

Fabrizio Rodríguez

Tecnología seleccionada: GitHub Actions

1. Problema que resuelve

Se explicará qué puede ocurrir cuando un equipo modifica el código de una aplicación y no comprueba si los cambios afectan funciones que ya trabajaban correctamente. Se presentará GitHub Actions como una herramienta que permite automatizar pruebas y otras tareas para detectar problemas con mayor anticipación.

2. ¿Qué es GitHub Actions?

Se investigará cómo GitHub Actions permite automatizar tareas dentro de un repositorio a partir de eventos, como subir cambios o abrir una solicitud de integración. Se explicará que las instrucciones se definen en archivos de configuración y que la herramienta puede ejecutar pruebas, verificaciones y procesos de despliegue.

3. ¿Cómo funciona y cuáles son sus componentes?

Se describirá el flujo básico de trabajo: ocurre un evento, se activa un workflow, se ejecutan los jobs y steps configurados en un runner y se obtiene un resultado. Se explicará cada componente con ejemplos sencillos, sin profundizar en configuraciones avanzadas.

4. Ejemplo práctico: MiniAPI de pedidos

Se utilizará una aplicación pequeña que simule la validación de pedidos, con una regla sencilla: la cantidad solicitada debe ser mayor que cero. Se prepararán pruebas automáticas para comprobar el comportamiento esperado y un flujo de GitHub Actions que las ejecute cuando se suban cambios al repositorio.

El ejemplo mostrará dos situaciones: una prueba que falla debido a un error introducido intencionalmente y una ejecución posterior que termina correctamente después de corregirlo. Se utilizarán capturas auténticas de ambas ejecuciones para explicar qué detectó el sistema y qué información recibe el equipo.

5. Relación con las tecnologías estudiadas

Se comparará GitHub Actions con las herramientas y conceptos trabajados durante el curso:

- **Git y GitHub:** permiten gestionar versiones y colaborar; GitHub Actions automatiza tareas relacionadas con los cambios del repositorio.
- **API y GraphQL:** las operaciones de una aplicación pueden someterse a pruebas automatizadas para comprobar su comportamiento.
- **PostgreSQL:** las pruebas pueden verificar operaciones que interactúan con la base de datos cuando el entorno está preparado para ello.
- **Docker:** puede utilizarse para ejecutar aplicaciones o servicios en contenedores durante un flujo de trabajo.
- **Autenticación y permisos:** pueden comprobarse mediante pruebas, pero GitHub Actions no sustituye los mecanismos de seguridad de la aplicación.

Se destacará que estas tecnologías tienen funciones diferentes y que pueden complementarse dentro de un proyecto de backend.

6. Ventajas, costos y limitaciones

Se analizarán las ventajas de automatizar tareas repetitivas, detectar errores antes de integrar cambios y obtener resultados consistentes. También se explicará que configurar y mantener los flujos requiere tiempo, que las pruebas deben cubrir los casos relevantes y que las ejecuciones consumen recursos sujetos a las condiciones del servicio utilizado.

Se aclarará que una ejecución exitosa solo confirma que las comprobaciones configuradas pasaron; no garantiza que la aplicación esté libre de errores.

7. ¿Cuándo no conviene utilizarla?

Se explicará que no siempre es necesario crear flujos complejos para proyectos pequeños que todavía no tienen pruebas ni tareas repetitivas. También se señalará que una automatización mal configurada o con pruebas insuficientes puede generar una falsa sensación de seguridad.

La decisión de utilizarla debe considerar las necesidades del proyecto, el tiempo de mantenimiento y el beneficio de automatizar el proceso.

8. Propuesta para la exposición

La exposición se organizará alrededor de una pregunta: ¿cómo puede un equipo detectar que un cambio rompió una función antes de que el problema llegue al usuario?

Se presentará el caso de la MiniAPI de pedidos mediante un esquema del flujo de trabajo y dos capturas de ejecuciones reales, una fallida y otra exitosa. Las diapositivas tendrán poco texto y se centrarán en explicar el problema, el funcionamiento, los resultados y las limitaciones de la herramienta.

La duración prevista será de 12 a 15 minutos, con una distribución del contenido entre los tres integrantes.

9. Fuentes de consulta

1. GitHub Docs. *Understanding GitHub Actions*. https://docs.github.com/en/actions/get-started/understand-github-actions
2. GitHub Docs. *Building and testing your code*. https://docs.github.com/en/actions/tutorials/build-and-test-code
3. GitHub Docs. *Using secrets in GitHub Actions*. https://docs.github.com/en/actions/security-for-github-actions/security-guides/using-secrets-in-github-actions

Estas fuentes se consultarán para fundamentar la explicación de los componentes, la automatización de pruebas y las consideraciones de seguridad. Se revisarán durante la investigación y se incorporarán al reporte final.